# NEON ESCAPE

Jogo 2D de sobrevivência em Python + Pygame

Computação Gráfica · Pedro Henrique Chaves

# Objetivo e regras

Chegue a 1000 pontos ou sobreviva 60 segundos. Zerar vidas ou energia é Game Over.

### Núcleo verde

+100 pontos e +20 de energia

### Núcleo amarelo

+250 pontos

### Drone inimigo

−1 vida, −30 de energia e 1,2 s de invulnerabilidade

### Progressão

Nova fase a cada 15 s, até a fase 8

# Arquitetura do código

| Arquivo | Responsabilidade |
| --- | --- |
| main.py | Ponto de entrada: inicia o jogo |
| settings.py | Constantes: tela, cores, metas |
| game.py | Estados, fases, colisões e cenário |
| player.py | Movimento, vidas, energia e desenho |
| enemy.py | Drones inimigos, mais rápidos a cada fase |
| collectible.py | Núcleos de energia e bônus |
| particles.py | Sistema de partículas |
| ui.py | Menu, HUD e mensagens |

# Máquina de estados game.py

```
class Game:
  MENU = "menu"
  INSTRUCTIONS = "instructions"
  PLAYING = "playing"
  PAUSED = "paused"
  GAME_OVER = "game_over"
  VICTORY = "victory"

  def update(self, dt):
    if self.state ≠ self.PLAYING:
      return
```

- Um único atributo, state, decide o que o jogo atualiza e desenha.
- handle_event troca de estado conforme a tecla: ENTER, ESC, M e I.
- A lógica só roda em PLAYING, então pausar congela o jogo sem código extra.

# Fases e colisão game.py

```
new_level = min(8,
  1 + int(elapsed // 15))

if enemy.rect.colliderect(
    player.rect):
  if player.hit():
    particles.burst(
      center, MAGENTA, 18)
```

- A fase é o tempo decorrido dividido por 15, limitada a 8.
- Cada entidade tem um pygame.Rect; colliderect testa a interseção.
- hit() devolve True só fora da invulnerabilidade, então a explosão de partículas ocorre uma vez por dano.

# O jogador player.py

```
direction = pygame.Vector2(
  dx, dy).normalize()
rect.x += round(
  direction.x * speed)
rect.clamp_ip(area_jogavel)

pulse = 1 + 0.10 * sin(t * 7)
size = int(18 * pulse)
```

- normalize() impede que a diagonal seja mais rápida.
- clamp_ip mantém a nave dentro da área jogável.
- O seno gera o pulso de escala do desenho.
- Após o dano, 1,2 s de invulnerabilidade com piscar.

# Computação Gráfica no código

| Técnica | Onde aparece |
| --- | --- |
| Primitivas 2D | pygame.draw: círculos, retângulos, linhas, polígonos |
| Animação por tempo | dt acumulado em elapsed e anim_time |
| Escala | Pulso por seno no jogador |
| Paralaxe | Grade e prédios com deslocamentos distintos |
| Partículas | burst() em coleta, dano e nova fase |
| Colisão | pygame.Rect e colliderect |

```
python -m pip install -r requirements.txt
python main.py
```